

Criando uma *landing page*

Uma sessão completa com o Codex, do diretório vazio ao primeiro commit. Vamos criar a landing page do **FocoFlow**, um app fictício de foco, aplicando os comandos das aulas 1 e 2: `/init`, `/plan`, `/diff` e `/review`.

Briefing → prompt

`/init` · AGENTS.md

`/plan`

`/diff` · `/review`

Commit

O briefing do *FocoFlow*

Todo bom projeto com IA começa com clareza sobre o que se quer. Antes de abrir o Codex, escreva o briefing: para quem é a página, o que ela precisa ter e com quais tecnologias.

PRODUTO

FocoFlow

App (fictício) que organiza o dia em blocos de foco, bloqueia distrações e gera relatórios de produtividade.

PÚBLICO

Estudantes e profissionais

Quem trabalha com a técnica Pomodoro e quer medir quanto realmente produziu.

OBJETIVO

Converter em download

Apresentar o app, gerar confiança com depoimentos e levar ao botão "Baixar grátis".

SEÇÕES PEDIDAS

- ✓ Cabeçalho com logo, menu e botão
- ✓ Hero com título, subtítulo e chamadas
- ✓ Três recursos principais
- ✓ "Como funciona" em 3 passos
- ✓ Depoimentos e chamada final

RESTRIÇÕES TÉCNICAS

HTML + CSS puros, sem framework e sem build. Assim o foco fica no fluxo com o Codex, não em ferramentas.

Sobre esta aula: as saídas do terminal marcadas como **SIMULAÇÃO** são ilustrativas, e a sua sessão vai variar. Os **arquivos, números e prints são reais**: o projeto está na pasta `focoflow/` desta aula.

Preparando o projeto e o *AGENTS.md*

Crie a pasta, inicie o Git (checkpoints salvam vidas) e abra o Codex com permissão de escrita no workspace. Em seguida, use `/init` para registrar as regras do projeto.

TERMINAL

SIMULAÇÃO

```
$ mkdir focoflow && cd focoflow
$ git init
Initialized empty Git repository in .../focoflow/.git/
$ codex -s workspace-write -a on-request

> /init
• Analisando o diretório... (vazio)
• Perguntei sobre a stack e o objetivo do projeto
✓ AGENTS.md criado (20 linhas)
```

Diretório vazio? Diga no mesmo prompt qual é a stack desejada. O `/init` usa isso para escrever as convenções.

FOCOFLOW/AGENTS.MD (ARQUIVO REAL, TRECHO)

AGENTS.md – FocoFlow (landing page)

Stack

- HTML5 + CSS3 puros, sem frameworks e sem build.
- Arquivos: index.html e styles.css.

Convenções

- Todo o texto da página em português do Brasil.
- Cores e espaçamentos como variáveis CSS em :root.
- Classes em kebab-case (.hero-title, .feature-card).
- HTML semântico: header, main, section, footer.

Como validar

- python3 -m http.server 8000
- Conferir a página em largura de desktop (1280px).

Escrevendo o *prompt*

Um bom prompt para o front-end tem seis partes: objetivo, público, estrutura, stack, estilo e critério de pronto. Quanto mais concreto, menos idas e vindas.

Objetivo: crie a landing page do FocoFlow, um app de foco e produtividade, para levar o visitante a baixar o app.

Público: estudantes e profissionais que usam Pomodoro.

Estrutura: cabeçalho (logo, menu, botão "Baixar grátis"), hero com título forte + card ilustrando um timer, 3 recursos, "Como funciona" em 3 passos, 2 depoimentos, CTA final e rodapé.

Stack: index.html + styles.css, seguindo o AGENTS.md.

Estilo: moderno e limpo, roxo (#5b4cf0) como cor de marca, fonte Plus Jakarta Sans, cantos arredondados e sombras suaves.

Pronto quando: a página abrir sem erros no navegador e todas as seções estiverem visíveis em 1280px.

1 Objetivo + público

Orientam o tom do texto e a hierarquia.

2 Estrutura

Evita seções esquecidas ou inventadas.

3 Stack + estilo

Reforçam o AGENTS.md com o visual desejado.

4 Critério de pronto

Diz ao agente como validar o próprio trabalho.

Repare no critério de pronto: ele só fala de **1280px**. Guarde isso, porque vai fazer diferença no `/review`.

Alinhando o plano com */plan*

Em vez de mandar o prompt direto, use `/plan`. O Codex propõe o caminho antes de escrever código, e corrigir um plano custa bem menos que corrigir código.

SESSÃO

SIMULAÇÃO

› `/plan` [cola o prompt da página anterior]

Plano proposto

1. `index.html`: estrutura semântica com `header`, `main` (`hero`, `recursos`, `como funciona`, `depoimentos`, `cta`) e `footer`, com âncoras `#recursos` e `#como-funciona`
2. `styles.css`: variáveis de cor e raio em `:root`, fonte Plus Jakarta Sans e botões `primário/ghost/claro`
3. Hero em grid de 2 colunas; card do timer feito só com CSS (`conic-gradient`), sem imagens
4. Seções em grid de 3 colunas; CTA com gradiente
5. Validar servindo com `python3 -m http.server`

› Pode seguir. Use emojis simples como ícones dos recursos.

O QUE OBSERVAR NUM PLANO

- Seguiu a **stack** do AGENTS.md?
- Cobriu **todas as seções** do briefing?
- Inventou algo que você não pediu?
- Explicou **como vai validar**?

Aproveite a resposta ao plano para ajustar detalhes, como a escolha dos ícones aqui. Ajustar um plano é muito mais barato que refazer um arquivo.

O Codex *trabalhando*

Com `workspace-write`, o Codex cria e edita arquivos na pasta do projeto sem pedir licença. Para **sair do limite**, como abrir uma porta de rede, ele para e pergunta.

SESSÃO

SIMULAÇÃO

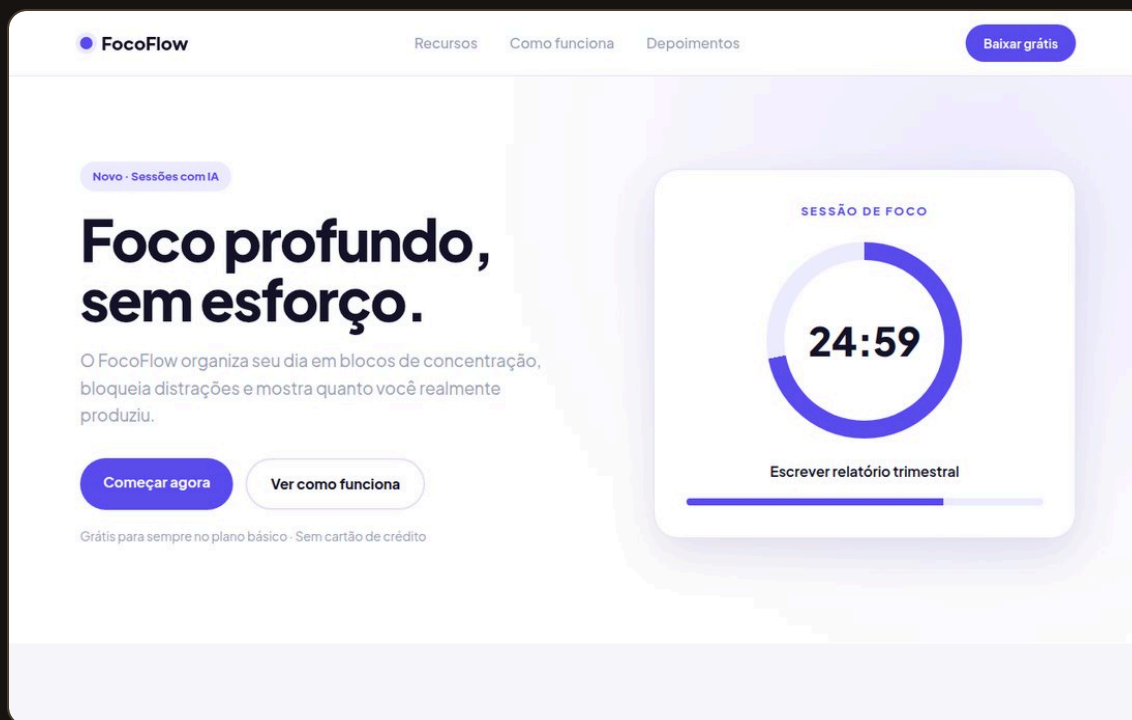
- Lendo AGENTS.md
 - ✓ Criado index.html +112 linhas
 - ✓ Criado styles.css +111 linhas
 - Conferindo âncoras e links internos... ok
- ⚠ Aprovação necessária
- O Codex quer executar:
- ```
python3 -m http.server 8000
```
- (abre uma porta de rede local)
- [Enter] aprovar [Esc] recusar
- Servidor em http://localhost:8000
  - Página carregada sem erros no console
  - ✓ Pronto: landing page criada em 2 arquivos.

### ENQUANTO ELE TRABALHA

- `Enter` injeta uma instrução nova **na rodada atual**
- `Tab` enfileira uma mensagem para a **próxima rodada**
- `/ps` mostra o servidor rodando em segundo plano

Prefere você mesmo subir o servidor? Use `!python3 -m http.server 8000` no composer: o `!` roda o comando no shell sem sair do Codex.

## A primeira versão *no navegador*



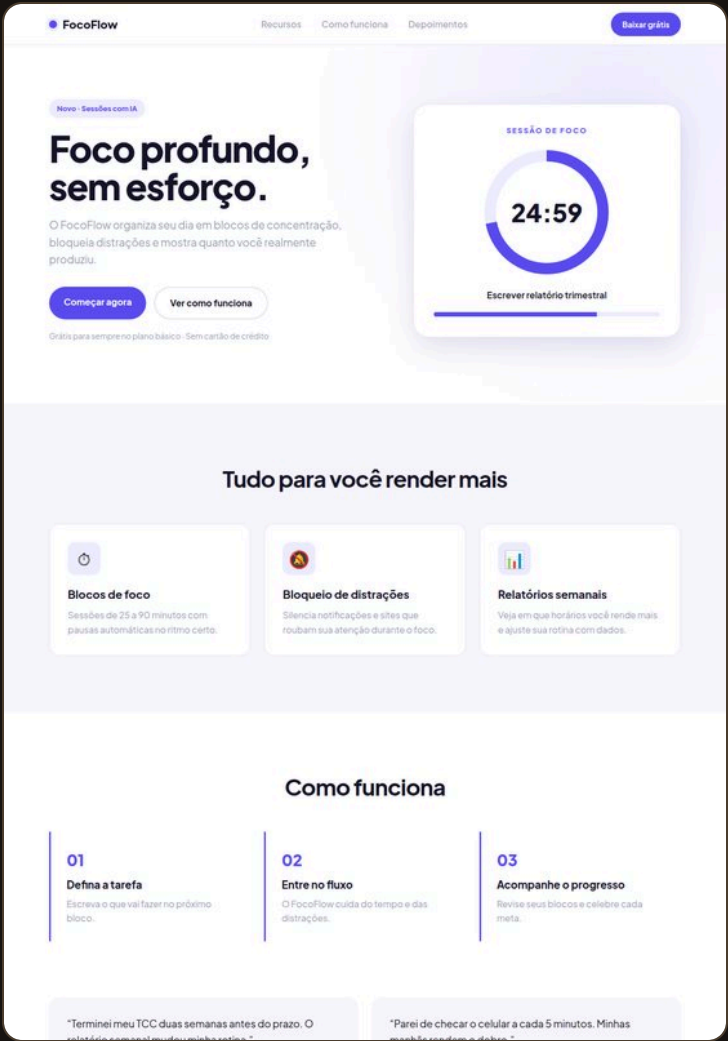
Print real · index.html v1 em 1280 × 800

### O QUE FICOU BOM

- ✓ Hierarquia clara: título, subtítulo e CTA
- ✓ Card do timer feito só com CSS
- ✓ Cores de marca em variáveis
- ✓ HTML semântico, como pedia o AGENTS.md

Em **uma rodada**, o Codex entregou uma página completa e coerente. O resultado depende muito do briefing e do plano que vieram antes.

# A página *completa*



Print real · página inteira (v1)

| SEÇÃO         | ELEMENTO               | TÉCNICA                                     |
|---------------|------------------------|---------------------------------------------|
| Cabeçalho     | Logo, menu, botão      | Sticky + blur                               |
| Hero          | Título + card do timer | Grid 2 colunas, <code>conic-gradient</code> |
| Recursos      | 3 cards com ícone      | Grid 3 colunas                              |
| Como funciona | 3 passos numerados     | CSS <code>counter()</code>                  |
| Depoimentos   | 2 citações             | <code>blockquote</code> + <code>cite</code> |
| CTA final     | Chamada + botão        | Gradiente linear                            |

Foram 2 arquivos e 223 linhas de código, mais as 20 do AGENTS.md. Tudo está na pasta `focoflow/` desta aula, e dá para abrir o `index.html` direto no navegador.

## Conferindo as mudanças com */diff*

Antes de aceitar o trabalho, veja exatamente o que foi criado. O `/diff` mostra também os arquivos que o Git ainda não rastreia, que aqui são todos.

SESSÃO

SIMULAÇÃO

```
> /diff
```

```
Arquivos novos (não rastreados):
```

```
A AGENTS.md +20
```

```
A index.html +112
```

```
A styles.css +111
```

```
3 arquivos · 243 inserções
```

### O QUE PROCURAR NO DIFF

- Arquivos **fora do combinado** (algum a mais?)
- Valores **fixos em px** onde deveria ser fluido
- Textos, links e nomes de classe coerentes
- Segredos ou dados sensíveis (nunca devem aparecer)

STYLES.CSS (TRECHO REAL)

```
:root {
 --brand: #5b4cf0;
 --ink: #16142b;
 --muted: #9aa0b4;
 --bg: #ffffff;
}

.container { width: 1120px; margin: 0 auto; }
.hero-grid { display: grid;
 grid-template-columns: 1.1fr 0.9fr; }
```

Viu o `width: 1120px` fixo no `.container` e as grids sem media query?  
No desktop tudo parece perfeito, mas isso vai quebrar no celular.

## A segunda opinião do */review*

O `/review` analisa a working tree como um revisor humano faria. Os achados abaixo são **reais**: conferimos cada um na página.

ACHADOS DO `/REVIEW`

FORMATO SIMULADO

P1

### Página quebra no celular

`.container` com largura fixa de 1120px e grids sem media queries: em 390px o conteúdo sai da tela e cria rolagem horizontal.

P2

### Contraste insuficiente

O texto secundário ( `#9aa0b4` sobre branco) tem contraste de **2,61:1**, abaixo do mínimo AA de 4,5:1.

P2

### Ícones sem tratamento de acessibilidade

Os emojis decorativos são lidos em voz alta por leitores de tela, e falta um link "pular para o conteúdo".

P3

### SEO básico

Falta `<meta name="description">` para os resultados de busca.



Print real · v1 em 390px: o conteúdo é cortado

## Salvando o ponto de partida com *commit*

Os problemas do review ficam para a próxima aula. Antes, registre esta versão: se algum ajuste der errado, você volta para cá com um comando.

SESSÃO

SIMULAÇÃO

```
> !git add .
> !git commit -m "feat: landing page FocoFlow v1"
[main (root-commit) 3f9c2a1] feat: landing page FocoFlow v1
3 files changed, 243 insertions(+)

> /rename landing-page-focoflow
✓ Sessão renomeada

> /quit
```

Prefere revisar fora da sessão? `codex review --uncommitted` faz a mesma revisão direto no terminal, antes do commit (aula 1).

### POR QUE COMMITAR AGORA

- ✓ Cria um **checkpoint** antes das mudanças grandes
- ✓ Separa o trabalho do Codex em etapas revisáveis
- ✓ `git diff` passa a mostrar só o que mudar a partir daqui

### NOMEAR A SESSÃO

Com `/rename`, a próxima aula começa com `codex resume` e você encontra a sessão pelo nome, com todo o contexto.

## O que esta sessão *ensinou*

1

### Briefing antes do prompt

Saber o que se quer é metade do trabalho, com IA ou sem ela.

2

### AGENTS.md como contrato

As convenções valem para todas as rodadas, sem repetir no prompt.

3

### Planejar é barato

`/plan` corrige o rumo antes de qualquer linha de código.

4

### O critério de pronto define a qualidade

Pedimos só 1280px e recebemos só 1280px.

5

### Revisar sempre

`/diff` mostra o que mudou, e `/review` mostra o que você não viu.

6

### Commit como checkpoint

Um commit por etapa deixa cada mudança reversível.

**A grande lição:** o agente faz exatamente o que o critério de pronto pede. Se você quer uma página responsiva e acessível, **diga isso no prompt e no AGENTS.md.**

## Teste *rápido*

1. Por que usar `/plan` antes de pedir a landing page?

A) Para gastar menos tokens

B) Para validar o caminho antes de haver código

C) Porque é obrigatório

D) Para gerar o AGENTS.md

**B.** Corrigir um plano custa muito menos que refazer arquivos já escritos.

2. Com `-s workspace-write -a on-request`, quando o Codex pediu aprovação?

A) Ao criar o index.html

B) Ao editar o styles.css

C) Ao abrir um servidor na porta 8000

D) Em todo comando

**C.** Criar e editar arquivos no workspace está dentro do limite. Abrir uma porta de rede sai dele, então o Codex pede aprovação.

3. Por que a v1 quebrou no celular?

A) Bug do Codex

B) O critério de pronto só pedia 1280px

C) Falta de JavaScript

D) A fonte não carregou

**B.** O agente validou exatamente o que foi pedido. A responsividade precisa estar no prompt e no AGENTS.md.

4. Qual comando mostra os arquivos criados, inclusive os que o Git ainda não rastreia?

A) `/status`

B) `/review`

C) `/diff`

D) `/copy`

**C.** O `/diff` mostra as mudanças staged, as unstaged e os arquivos não rastreados.

○ RECAPITULANDO

# Do briefing ao commit *em uma sessão*

Escrevemos o briefing, registramos as regras no `AGENTS.md` com `/init`, alinhamos o plano com `/plan`, deixamos o Codex construir, conferimos com `/diff` e `/review` e salvamos um checkpoint. A página ficou ótima no desktop, e o review mostrou o que falta para o celular e para a acessibilidade.

Briefing

`/init`

`/plan`

`/diff · /review`

`git commit`

Próxima aula → Ajustes na landing page

Projeto real desta aula: `focoflow/` (`AGENTS.md`, `index.html`, `styles.css`). As saídas de terminal marcadas como simulação são ilustrativas; os prints, as contagens de linhas e as medidas de contraste foram obtidos dos arquivos reais.